MDPI

Article

Enhancing Rule Construction in Grammatical Evolution with Advanced Crossover Techniques

Ioannis G. Tsoulos[1]*, George Georgoulas[2] and Vasileios Charilogis[3]

[1] Department of Informatics and Telecommunications. University of Ioannina, Greece; itsoulos@uoi.gr
[2] Department of Mechanical Engineering and Aeronautics, University of Patras, Panepistimioupoli Patron, Rio 26504, Greece; georgoul@gmail.com
[3] Department of Informatics and Telecommunications. University of Ioannina, Greece; charilog@uoi.gr
* Correspondence: itsoulos@uoi.gr

Abstract

Many real-world problems can be formulated as classification tasks in machine learning and addressed using a wide range of advanced algorithms developed for this purpose. Among these approaches, methods based on Grammatical Evolution (GE) can also be employed, such as GenClass, a technique designed to automatically construct classification rules. Although GenClass has been successfully applied to a variety of classification problems, its evolutionary search process may suffer from limitations such as entrapment in local minima of the error function and premature convergence, which can prevent the method from discovering high-quality classification rules. The present work proposes an enhanced version of GenClass through the introduction of advanced crossover techniques operating on the integer-valued chromosomes employed by the method, with the aim of achieving a significant reduction in classification error. The proposed crossover operators follow two different strategies. Some of them are purely stochastic, introducing greater diversity into the evolutionary search, whereas others exploit information already generated during the Grammatical Evolution process to perform targeted exchanges of genetic material between chromosomes. In this way, the latter operators attempt to guide the evolutionary search toward more promising regions of the search space while preserving useful information discovered during previous generations. The proposed techniques were evaluated on a diverse collection of real-world classification datasets, including several problems characterized by a large number of patterns and/or a large number of classes. The experimental results demonstrate that the proposed crossover strategies can substantially improve the performance of the original method, leading to a noticeable reduction in classification error across a wide range of classification problems.

Keywords: Grammatical Evolution; Classification; Crossover Operators; Evolutionary Algorithms; Machine Learning.

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Journal Not Specified* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Genetic Algorithms (GAs) are stochastic optimization methods whose foundations can be traced back to the pioneering work of Holland [1]. They constitute an important class of evolutionary computation techniques [2], drawing inspiration from the principles of natural evolution. A typical Genetic Algorithm starts with a population of candidate solutions to the optimization problem under consideration. This population is progressively improved over successive generations through evolutionary operators such as selection,

crossover, and mutation [3–5]. These operators enable the exploration of the search space while promoting the propagation of promising solutions throughout the population. Due to their flexibility and ability to address complex optimization problems without requiring explicit gradient information, Genetic Algorithms have been successfully applied across a broad range of domains. Representative applications include network optimization [6], robotics [7,8], energy-related problems [9,10], medical applications [11], and agricultural problems [12], among many others.

Grammatical Evolution (GE) [13] is a specialized form of Genetic Algorithms that employs integer-valued chromosomes. Each chromosome consists of a sequence of integer codons that are used to select among the production rules defined by a given Backus–Naur Form (BNF) grammar [14]. Through a genotype-to-phenotype mapping process, these codons guide the derivation of syntactically valid expressions or programs. The primary objective of Grammatical Evolution is therefore to evolve programs or expressions in a target language whose syntax is specified by the underlying grammar.

Grammatical Evolution has been successfully applied to a wide range of real-world problems across diverse scientific and engineering domains. Representative applications include data fitting [15,16], credit classification [17], network intrusion detection [18], the solution of differential equations [19], drinking water quality monitoring [20], and the automatic construction of optimization methods [21]. GE has also been employed in trigonometric problems [22], automatic music composition [23], neural network design [24,25], and computer games [26,27]. Additional applications can be found in energy demand modeling [28], combinatorial optimization [29], security and cryptography [30], automatic decision tree construction [31], analog circuit design [32], taxonomy discovery from Wikipedia [33], algorithmic trading [34], bioinformatics [35], and human glycemia modeling [36]. The diversity of these applications demonstrates the flexibility of Grammatical Evolution as a general-purpose evolutionary framework for addressing complex problems in different domains.

Among the techniques that have been proposed based on Grammatical Evolution is a recently introduced rule construction method [37], for which a software implementation [38] is also publicly available through the application's website https://github.com/itsoulos/GenClass/(accessed on 2 October 2026). This method automatically constructs programs for data classification using a representation that closely resembles the C programming language. Consequently, the generated classification programs are human-readable and can be easily inspected and interpreted, providing a considerable advantage in applications where model interpretability is important. The rule construction method has been successfully applied to a variety of real-world classification problems, including pollution-related applications[39] and network-related problems [40], among others.

The aforementioned method employs Grammatical Evolution to evolve classification programs that minimize the classification error on a given dataset. More specifically, the classification error can be defined as

$$E_C(C) = 100 \times \frac{1}{M} \sum_{i=1}^{M} \mathbb{I}(C(\mathbf{x}_i) \neq y_i) \quad (1)$$

where the corresponding training set is defined as $T = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_M, y_M)\}$. Here, M denotes the number of patterns in the training set, $\mathbf{x}_i$ represents the input feature vector of the (i)-th pattern, and y_i denotes its corresponding target class label. The function $C(\mathbf{x}_i)$ represents the class predicted by the classification program generated through Grammatical Evolution, while $(\mathbb{I}(\cdot))$ is the indicator function, which takes the value 1 when its argument is true and 0 otherwise. Therefore, $E_C(C)$ expresses the percentage of training patterns that are incorrectly classified by the generated program.

Although the aforementioned technique has demonstrated particularly promising performance, even when applied to challenging data classification problems, it still exhibits several limitations that may adversely affect the effectiveness of the evolutionary search process. One of the most important issues is premature convergence, where the population rapidly loses genetic diversity and converges before sufficiently exploring the search space. As a consequence, potentially useful genetic material may disappear from the population, reducing the ability of the evolutionary process to discover alternative and potentially more effective classification rules.

Another important limitation is the tendency of the evolutionary search to become trapped in local minima of the classification error function, or, more generally, to stagnate in suboptimal regions of the search space. In such cases, the evolutionary operators may fail to generate sufficiently diverse or beneficial modifications to the existing chromosomes, preventing the algorithm from escaping these regions and discovering solutions with lower classification error. These limitations become particularly important in complex classification problems characterized by large or highly irregular search spaces, where effective exploration and preservation of population diversity are essential for obtaining high-quality classification rules. Consequently, improving the search capabilities of the method and enhancing the exchange of useful genetic information among candidate solutions constitute important directions for further improving its classification performance.

In the present work, advanced crossover techniques are introduced in order to alleviate the aforementioned limitations and improve the search capabilities of the original method. These techniques are applied at each generation to a relatively small number of chromosomes selected randomly from the genetic population. Restricting their application to a subset of the population allows the proposed operators to enhance the exploration of the search space without substantially increasing the computational cost of the evolutionary process or excessively disrupting the population structure.

The randomly selected chromosomes are subjected to additional crossover operations designed to generate new combinations of genetic material and, consequently, potentially more effective classification programs. The introduction of such combinations can increase population diversity and provide the evolutionary process with additional opportunities to escape from suboptimal regions of the search space. This mechanism is particularly important when the population has begun to converge and the conventional evolutionary operators are no longer able to produce sufficiently diverse candidate solutions.

Furthermore, the random selection of chromosomes ensures that the additional crossover operations are not systematically restricted to a particular subset of the population. In this way, different candidate solutions can participate in the enhanced crossover process during successive generations, while the majority of the population continues to evolve according to the standard evolutionary procedure. The proposed strategy therefore attempts to achieve a balance between exploration and exploitation: it introduces additional diversity and explores new combinations of genetic material, while preserving the evolutionary information already accumulated by the population.

In the present work, three crossover strategies are considered: **Standard**, **Targeted** and **TargetedWorst** crossover. Standard crossover follows a purely stochastic strategy, in which the exchanged chromosome regions are selected without exploiting information obtained from the evaluation of the generated classification programs. Targeted crossover, on the other hand, makes use of information collected during the evolutionary process to identify chromosome regions where crossover may be more beneficial. TargetedWorst extends this idea by focusing the crossover operation on regions associated with poorly performing components of the corresponding classification program. Thus, the three strategies represent progressively more informed crossover mechanisms, ranging from

purely random recombination to crossover operations guided by the performance of the evolved solutions. A detailed description of each strategy is provided in the following subsections.

The remainder of this article is organized as follows. Section 2 introduces the original rule construction method and describes the proposed crossover techniques in detail. Section 3 presents the classification datasets used in the experimental evaluation, followed by the experimental results and the corresponding statistical analysis. Finally, Section 4 summarizes the main findings of the study and outlines the conclusions drawn from the experimental evaluation.

2. Materials and Methods

This section first provides a detailed description of the rule construction method and subsequently introduces the proposed crossover techniques designed to enhance its evolutionary search process.

2.1. The rule construction method

The rule construction method, hereafter referred to as GenClass, generates classification rules expressed in a C-like language according to the BNF grammar presented in Algorithm 1. The grammar shown in the algorithm is an extended representation of the original BNF grammar, in which each alternative production rule associated with a non-terminal symbol is assigned a corresponding production index. These indices are subsequently used during the genotype-to-phenotype mapping process to determine the production rules selected by the integer-valued chromosomes of GenClass.

Algorithm 1 BNF grammar employed by the GenClass method.

```
<S> ::= if (<BEXPR>) CLASS=0 else CLASS=1                    (0)

<BEXPR> ::= <VARIABLE><BOOLOP><EXPR>                         (0)
          | !(<BEXPR>)                                       (1)
          | <VARIABLE><BOOLOP><EXPR>&<BEXPR>                 (2)
          | <VARIABLE><BOOLOP><EXPR>|<BEXPR>                 (3)

<BOOLOP> ::= >                                               (0)
           | >=                                              (1)
           | <                                               (2)
           | <=                                              (3)

<EXPR> ::= (<EXPR><BINARYOP><EXPR>)                          (0)
         | <FUNCTION>(<EXPR>)                                (1)
         | <TERMINAL>                                        (2)

<BINARYOP> ::= +                                             (0)
             | -                                             (1)
             | *                                             (2)
             | /                                             (3)

<FUNCTION> ::= sin | cos | exp | log                        (0-3)

<TERMINAL> ::= <VARIABLE>                                    (0)
             | <DIGITLIST>.<DIGITLIST>                       (1)
             | (-<DIGITLIST>.<DIGITLIST>)                    (2)

<VARIABLE> ::= x1 | x2 | ... | xD                          (0-D-1)

<DIGITLIST> ::= <DIGIT>                                      (0)
              | <DIGIT><DIGIT>                               (1)
              | <DIGIT><DIGIT><DIGIT>                        (2)

<DIGIT> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9    (0-9)
```

The genotype-to-phenotype mapping process employed by GenClass is illustrated in Figure 1. Each chromosome consists of a sequence of integer-valued codons, which are sequentially used to select production rules from the BNF grammar. Starting from the initial non-terminal symbol $\langle S \rangle$, the mapping process identifies the next non-terminal to be expanded and determines the corresponding production rule according to

$$r = c_i \bmod N,$$

where c_i denotes the current integer codon, N is the number of alternative production rules available for the corresponding non-terminal symbol, and r is the index of the selected production. The selected production is then applied and the mapping process proceeds to the next non-terminal symbol using the next chromosome codon. This procedure is repeated until no non-terminal symbols remain and a complete syntactically valid classification program is generated.

As an illustrative example, Figure 1 demonstrates the successive mapping of an integer chromosome to the production rules of Algorithm 1. For instance, when the non-terminal $\langle BEXPR \rangle$ is expanded, four alternative productions are available. Therefore, for a codon value of 7, production 7 mod 4 = 3 is selected. Similarly, the remaining codons are successively consumed to expand the resulting non-terminal symbols until the final classification rule is obtained. In this way, the integer-valued genotype is transformed into an executable and human-readable C-like classification program.

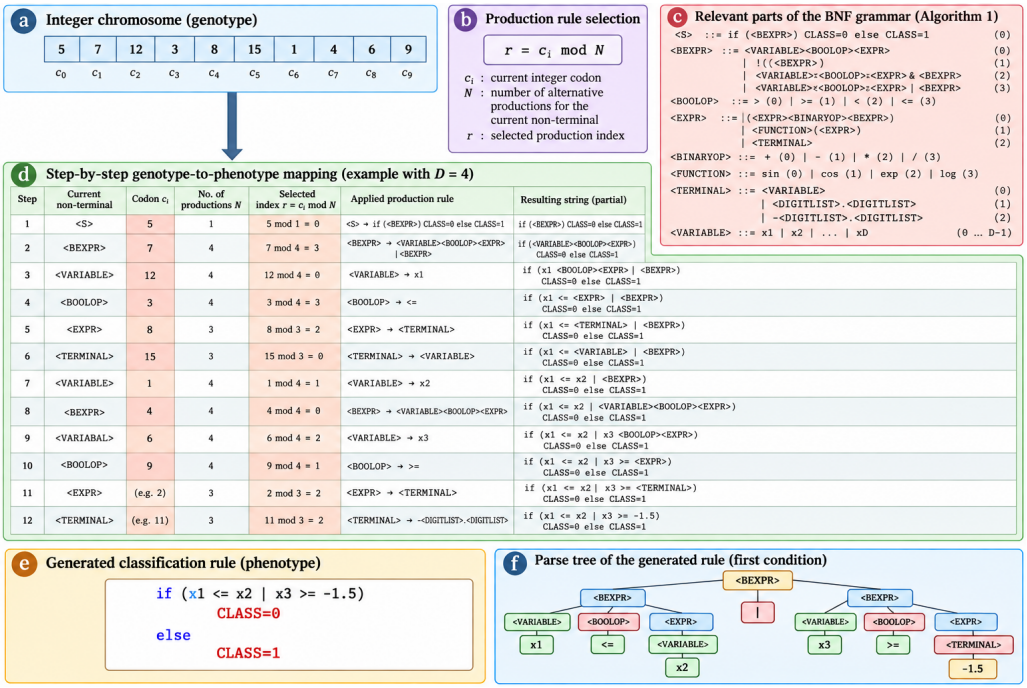


Figure 1. Illustration of the genotype-to-phenotype mapping process employed by GenClass. Integer-valued codons are sequentially mapped to BNF production rules using the modulo operation until a complete classification program is generated.

The main steps of the algorithm that produces classification programs are listed subsequently.

1. **Initialization step**.
   (a) **Initialize** the Genetic Algorithm parameters: N_c, the population size (number of chromosomes); N_g, the maximum number of generations; p_s, the selection rate; and p_m, the mutation rate.
   (b) **Generate** the initial population of N_c chromosomes, g_i, $i = 1, \ldots, N_c$, where each chromosome consists of a sequence of randomly generated integer-valued codons.
   (c) **Set** $t = 0$, the generation counter.
2. **Fitness calculation step**.
   (a) **For** $i = 1, \ldots, N_c$ **do**
      i. **Map** each chromosome g_i to its corresponding classification programC_i through the genotype-to-phenotype mapping process defined by the BNF grammar presented in Algorithm 1.
      ii. **Evaluate** the generated classification program C_i on the training set and compute its classification error. The resulting error value, denoted by f_i, is used as the fitness value of the corresponding chromosome g_i.
   (b) **End For**
3. **Application of genetic operations**.
   (a) **Selection**: Rank the chromosomes according to their fitness values in ascending order. The $(1 - p_s)N_c$ chromosomes with the lowest fitness values are directly transferred to the next generation without modification, while the remaining $p_s N_c$ chromosomes are replaced by new offspring generated through the crossover and mutation operators.

(b) **Crossover**: Generate pairs of offspring by applying the one-point crossover operator. For each pair of offspring, denoted as O_1 and O_2, two parent chromosomes, P_1 and P_2, are selected from the current population using tournament selection. A crossover point is then randomly selected along the chromosomes, and the genetic material located after this point is exchanged between the two parents. Consequently, O_1 inherits the first part of P_1 and the second part of P_2, whereas O_2 inherits the first part of P_2 and the second part of P_1. A graphical illustration of the one-point crossover operation is provided in Figure 2.

(c) **Mutation**: For each gene$g_{i,j}$ of chromosome g_i, a random number $r \in [0, 1]$ is generated. If $r \leq p_m$, where p_m denotes the mutation rate, the corresponding gene is replaced by a randomly generated integer value within the predefined range of admissible codon values. Otherwise, the gene remains unchanged.

4. **Termination check step**.

(a) **Set** $t = t + 1$

(b) **If** $t \leq N_g$ **then go to** Fitness Calculation step **else** terminate.

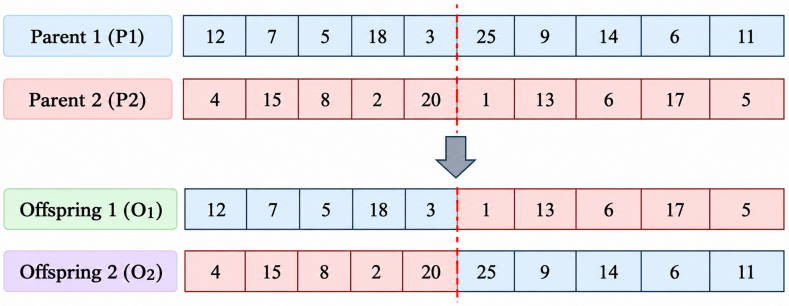


Figure 2. An example of the one - point crossover method.

2.2. The proposed crossover methods

In the present work, additional crossover techniques are introduced with the aim of improving the performance of GenClass and enhancing its ability to escape from local minima of the classification error function. These crossover operations are applied at each generation of the evolutionary process to a subset of the current population. More specifically, a number of chromosomes are randomly selected from the population and genetic material from each selected chromosome is exchanged with that of another randomly selected chromosome. In this way, new combinations of genetic material are introduced into the population, potentially increasing population diversity and allowing the evolutionary search to explore previously unvisited regions of the search space. The number of chromosomes subjected to these additional crossover operations at each generation is denoted by N_K throughout the remainder of this work.

The proposed procedure is incorporated into the evolutionary algorithm described in subsection 2.1 and is executed at each generation after the standard genetic operators of selection, crossover, and mutation have been applied. In this way, the additional crossover mechanism operates as an enhancement of the original evolutionary process without modifying its basic structure.

The proposed crossover techniques, denoted throughout this work as Standard, Targeted, and TargetedWorst, differ in the strategy employed to select and exchange genetic material between chromosomes. A detailed description of each technique is provided in the following subsections.

2.2.1. The Standard crossover method

In the Standard crossover technique, for each of the N_K selected chromosomes, denoted by g_c, a subset of k chromosomes is randomly selected from the current population,

$$S = \{g_{r_1}, g_{r_2}, \ldots, g_{r_k}\}.$$

The chromosome g_c is then successively combined with each chromosome $g_{r_i}, i = 1, \ldots, k$, using one-point crossover. For each pair (g_c, g_{r_i}), a crossover point is randomly selected and the corresponding genetic material is exchanged to produce a candidate offspring chromosome g_{new}. The fitness of g_{new} is subsequently evaluated using its classification error on the training set. If g_{new} achieves a lower fitness value than the current g_c, it replaces g_c; otherwise, the current chromosome remains unchanged. Importantly, whenever such an improvement occurs, the updated chromosome becomes the new g_c and is used in the subsequent crossover operation with $g_{r_{i+1}}$. Therefore, the procedure performs a sequential improvement of g_c, allowing beneficial genetic material obtained from successive crossover operations to be accumulated. After all k chromosomes in S have been examined, the final improved version of g_c is retained in the population.

The complete procedure of the Standard crossover technique is summarized in Algorithm 2. The method follows a greedy acceptance strategy, since a newly generated chromosome replaces the current chromosome only when it achieves a lower classification error. Furthermore, each accepted chromosome is immediately used in the subsequent crossover operation, allowing improvements obtained during successive crossover trials to be accumulated.

Algorithm 2 Standard crossover technique

Require: Population $G = \{g_1, g_2, \ldots, g_{N_c}\}$, number of selected chromosomes N_K, number of crossover trials k
Ensure: Updated population G

```
Randomly select N_K chromosomes from G
for each selected chromosome g_c do
    Randomly select a subset S = {g_r1, g_r2, ..., g_rk} from G
    for i = 1 to k do
        Randomly select a one-point crossover position
        Generate g_new by applying one-point crossover between g_c and g_ri
        Evaluate the fitness f(g_new)
        if f(g_new) < f(g_c) then
            g_c ← g_new
        end if
    end for
end for
return G
```

2.2.2. The Targeted crossover method

In the Targeted crossover technique, the exchange of genetic material is guided by information collected during the genotype-to-phenotype mapping process. More specifically, the mapping procedure records the positions of the active codons of each chromosome together with their semantic role in the generated classification program. These codons may correspond to different grammatical components, such as functions, binary operators, Boolean operators, variables, and constants.

For each selected chromosome g_c, the Targeted technique performs k crossover trials. At each trial, an active codon position is randomly selected as the target of the crossover operation. A donor chromosome g_r is then randomly selected from the best fraction ρ of the current population, and the recorded mapping information is used to identify an

active codon in g_r having the same semantic type as the selected codon of g_c. Instead of performing an unrestricted one-point crossover, the Targeted technique transfers a bounded block of genetic material between the corresponding positions. The size of this block is determined from the distances to the next active codons in the two chromosomes and is additionally restricted by a predefined maximum block size $B_{\max}$. In this way, the transferred genetic material is associated with similar grammatical components of the classification programs represented by the two chromosomes.

The resulting chromosome g_{new} is subsequently mapped to its corresponding classification program and its fitness is evaluated on the training set. If

$$f(g_{\text{new}}) < f(g_c),$$

the new chromosome replaces g_c; otherwise, the current chromosome remains unchanged. As in the Standard technique, an accepted improvement is immediately retained and the updated chromosome is used in subsequent crossover trials. The main difference from Standard crossover is therefore that the exchange of genetic material is not determined exclusively by randomly selected crossover points. Instead, information obtained during the mapping process is exploited to perform crossover between semantically related regions of the participating chromosomes. This targeted exchange is intended to increase the probability of producing meaningful modifications to the generated classification programs while preserving useful genetic structures already discovered during the evolutionary process. The complete procedure of the Targeted crossover technique is summarized in Algorithm 3.

Algorithm 3 Targeted crossover technique

Require: Population $G = \{g_1, g_2, \ldots, g_{N_c}\}$, number of selected chromosomes N_K, number of crossover trials k, elite fraction ρ, maximum block size $B_{\max}$
Ensure: Updated population G

Randomly select N_K chromosomes from G
$N_e \leftarrow \max(2, \lfloor \rho N_c \rfloor)$
for each selected chromosome g_c **do**
  for $i = 1$ to k **do**
    Obtain the codon trace of the current chromosome g_c
    Construct the set A_c of active semantic codons corresponding to functions, binary operators, Boolean operators, variables, and constants
    if A_c is empty **then**
      break
    end if
    Randomly select a target codon $a_c \in A_c$
    Let t_c be the semantic type of a_c
    Randomly select a donor chromosome g_r from the best N_e chromosomes, with $g_r \neq g_c$
    Obtain the codon trace of g_r
    Construct the set A_r of active codons in g_r having semantic type t_c
    if A_r is empty **then**
      continue
    end if
    Randomly select a compatible donor codon $a_r \in A_r$
    Determine L_c, the distance from a_c to the next active codon in g_c
    Determine L_r, the distance from a_r to the next active codon in g_r
    $L \leftarrow \min(L_c, L_r, B_{\max})$
    Create $g_{\text{new}} \leftarrow g_c$
    Copy L codons from g_r, starting at a_r, to the corresponding block of g_{new} starting at a_c
    Evaluate the fitness $f(g_{\text{new}})$
    if $f(g_{\text{new}}) < f(g_c)$ **then**
      $g_c \leftarrow g_{\text{new}}$
    end if
  end for
end for
return G

2.2.3. The TargetedWorst crossover method

The TargetedWorst crossover technique extends the Targeted approach by additionally exploiting information about the classification performance of the generated program. While Targeted crossover may select an active semantic codon from the chromosome without considering the performance of individual classes, TargetedWorst attempts to concentrate the crossover operation on the chromosome region associated with the class exhibiting the poorest classification performance. The motivation behind this strategy is to modify preferentially those parts of the generated classification program that are more likely to contribute to the observed classification errors, while leaving better-performing components less affected.

For each selected chromosome g_c, the corresponding classification program is evaluated and the class exhibiting the worst classification performance is identified. The genotype-to-phenotype mapping information is subsequently used to determine the active codons associated with the corresponding part of the classification program. An active codon is then selected from this region and its semantic type is determined. As in the Targeted technique, a donor chromosome g_r is selected from the elite fraction of the population, and an active codon having the same semantic type is identified in the donor chromosome.

This establishes semantically compatible positions between the two chromosomes and allows genetic material from a high-quality solution to be transferred specifically to a poorly performing region of g_c.

A bounded block of genetic material is subsequently transferred from the donor chromosome to g_c. As in Targeted crossover, the size of the transferred block is determined using the active-codon structure of the participating chromosomes and is restricted by the predefined maximum block size $B_{\max}$. The resulting chromosome g_{new} is then mapped to its corresponding classification program and evaluated on the training set. If

$$f(g_{\text{new}}) < f(g_c),$$

the candidate chromosome replaces g_c; otherwise, the modification is discarded. Accepted improvements are immediately retained and are therefore available to subsequent crossover trials.

Consequently, TargetedWorst introduces an additional level of guidance compared with Targeted crossover. Targeted determines the exchanged genetic material according to semantic information obtained during genotype-to-phenotype mapping, whereas TargetedWorst combines this information with class-specific classification performance in order to focus the search on chromosome regions associated with poorly performing parts of the generated classifier. The complete procedure is summarized in Algorithm 4.

Algorithm 4 TargetedWorst crossover technique

Require: Population $G = \{g_1, g_2, \ldots, g_{N_c}\}$, number of selected chromosomes N_K, number of crossover trials k, elite fraction ρ, maximum block size $B_{\max}$
Ensure: Updated population G

Randomly select N_K chromosomes from G
$N_e \leftarrow \max(2, \lfloor \rho N_c \rfloor)$
for each selected chromosome g_c **do**
  for $i = 1$ to k **do**
    Evaluate the class-specific performance of g_c
    Identify the worst-performing class w
    Let C be the number of classes
    $N_R \leftarrow C - 1$
    if $w < N_R$ **then**
      Determine the chromosome segment R_w associated with class w
    else
      Set R_w to the entire chromosome
    end if
    Obtain the codon trace of the current chromosome g_c
    Construct the set A_c of active semantic codons of g_c that belong to R_w
    if A_c is empty **then**
      continue
    end if
    Randomly select a target codon $a_c \in A_c$
    Let t_c be the semantic type of a_c
    Randomly select a donor chromosome g_r from the best N_e chromosomes, with $g_r \neq g_c$
    Obtain the codon trace of g_r
    if $w < N_R$ **then**
      Determine the donor segment $R_w^{(r)}$ associated with class w
    else
      Set $R_w^{(r)}$ to the entire donor chromosome
    end if
    Construct the set A_r of active codons in g_r that belong to $R_w^{(r)}$ and have semantic type t_c
    if A_r is empty **then**
      continue
    end if
    Randomly select a compatible donor codon $a_r \in A_r$
    Determine L_c, the distance from a_c to the next active codon in g_c
    Determine L_r, the distance from a_r to the next active codon in g_r
    $L \leftarrow \min(L_c, L_r, B_{\max})$
    Restrict L to remain within the corresponding class segments and chromosome boundaries
    Create $g_{\text{new}} \leftarrow g_c$
    Copy L codons from g_r, starting at a_r, to g_{new}, starting at a_c
    Evaluate the fitness $f(g_{\text{new}})$
    if $f(g_{\text{new}}) < f(g_c)$ **then**
      $g_c \leftarrow g_{\text{new}}$
    end if
  end for
end for
return G

Figure 3 presents the general flowchart of the proposed method. The process starts with the generation of the initial population of integer-valued chromosomes. Each chromosome is mapped to a classification program through the BNF grammar, and the program is evaluated on the training set, which gives the fitness of the chromosome. The standard

genetic operators of selection, one-point crossover and mutation are then applied to the population. After them, the additional crossover procedure is executed. A small number of chromosomes is randomly selected and, for each of them, a number of crossover trials is performed. In every trial, the technique in use, Standard, Targeted or TargetedWorst, produces a new candidate chromosome, which is mapped to a classification program through the grammar and evaluated. If the candidate achieves a lower error than the current chromosome, it replaces it, otherwise the current chromosome is kept. The process continues with the next trial until all the trials of all the selected chromosomes are completed. Finally, the termination criterion is checked. If it is not satisfied, the algorithm returns to the mapping and evaluation of the new population, otherwise it terminates and returns the best classification program.

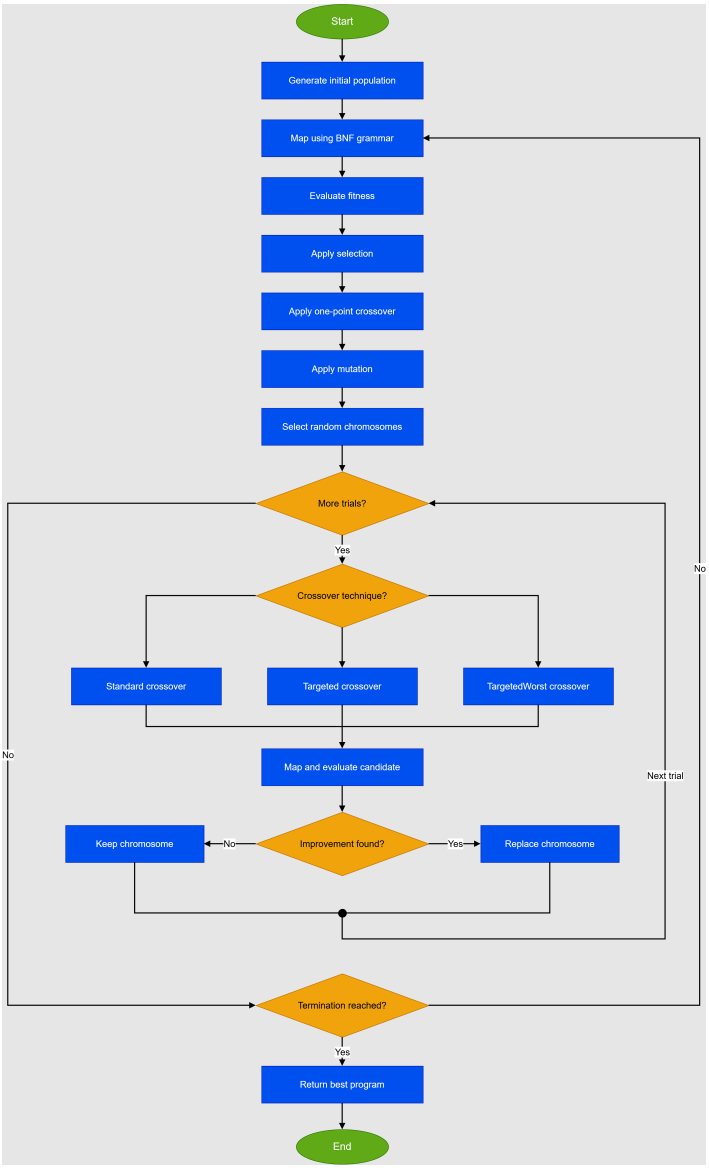


Figure 3. Flowchart of the proposed GenClass method with the additional crossover techniques.

3. Results

3.1. Experimental datasets

The datasets employed in the experimental evaluation were obtained from two widely used public repositories: the UCI Machine Learning Repository [41] and the KEEL Dataset Repository [42]. Both repositories provide a broad collection of benchmark datasets com-

monly used for the experimental evaluation and comparison of machine learning and classification algorithms. The datasets were accessed on 3 October 2026. The used datasets are the following:

1. **Alcohol.** This dataset contains data collected from experiments investigating patterns of alcohol consumption and is used as a classification benchmark in the present study [43].
2. **Appendicitis.** The Appendicitis dataset is a binary classification dataset containing 106 instances and 7 numerical features, used to predict the presence or absence of acute appendicitis based on clinical measurements [44].
3. The **Beed** dataset. The Bangalore EEG Epilepsy Dataset (BEED), presented by Banu et al. [45], is an EEG dataset developed for epileptic seizure detection and classification. The dataset contains 8,000 patterns, each represented by 16 features.
4. **Dermatology.** The Dermatology dataset [46] contains clinical and histopathological measurements used for the differential diagnosis of dermatological diseases.
5. **Gene**. The Gene dataset, also known as the Splice-junction Gene Sequences dataset, contains 3,175 DNA sequences represented by 120 binary features. The dataset is used for the classification of splice-junction boundaries into three classes: exon–intron (EI) boundaries, intron–exon (IE) boundaries, and sequences containing neither type of boundary [47].
6. **Hayes-Roth.** The Hayes-Roth dataset is an artificial classification dataset comprising three classes and was originally designed to evaluate the behavior of prototype-based classifiers. The hobby attribute was generated randomly and therefore acts as a noise-inducing feature in the classification problem [48].
7. **Hepatitis.** The Hepatitis dataset contains clinical information collected from patients affected by hepatitis and includes a combination of integer- and real-valued attributes describing various patient characteristics and medical measurements [49].
8. **HouseVotes.** The House Votes dataset contains voting records of members of the U.S. House of Representatives on a set of congressional issues. The classification task aims to distinguish representatives according to their political party affiliation based on their recorded votes [50].
9. **Ionosphere.** The Ionosphere dataset contains radar measurements collected from signals transmitted through the ionosphere. The classification task aims to distinguish between radar returns that provide evidence of structure in the ionosphere and those that do not [51,52].
10. **Lymphography** [53], which is a dataset with four distinct classes.
11. **Nursery.** The Nursery dataset was derived from a hierarchical decision model originally developed to rank applications for admission to nursery schools. The model was used in Ljubljana, Slovenia, during the 1980s, when high demand for nursery schools required an objective procedure for evaluating and explaining admission decisions. The corresponding classification task involves assigning nursery school applications to different priority categories based on a set of socioeconomic and family-related attributes [54].
12. **Optdigits**. The Optical Recognition of Handwritten Digits (Optdigits) dataset contains digitized representations of handwritten numerals obtained from optical scans. The corresponding classification task aims to recognize and assign each handwritten sample to one of the ten digit classes, ranging from 0 to 9 [55].
13. **Penbased**. The Pen-Based Recognition of Handwritten Digits dataset contains samples of handwritten digits collected from 44 writers. Each digit is represented using its (x, y) coordinate information, resampled to eight points and encoded as a fixed-length

feature vector of 16 attributes. The corresponding classification task aims to assign each sample to one of the ten digit classes, ranging from 0 to 9 [56].

14. **Phoneme**. The Phoneme dataset contains acoustic measurements extracted from speech signals. The corresponding classification task aims to distinguish between nasal and oral phonemes based on their acoustic characteristics [57].
15. **PirVision.** The PirVision dataset contains sensor measurements collected for occupancy detection. The corresponding classification task aims to determine occupancy status based on data acquired from passive infrared (PIR) sensing devices [58].
16. **Satimage**. The Satimage dataset contains multispectral satellite image data representing different types of terrain. The corresponding classification task aims to assign image regions to different land-cover classes based on their spectral characteristics [59].
17. **Segment.** The Segment dataset contains image regions characterized by a set of numerical features extracted from outdoor images. The corresponding classification task aims to assign each region to one of several image categories based on its visual characteristics [60].
18. **Shuttle.** The Shuttle dataset was originally developed to derive comprehensible classification rules for determining the conditions under which automatic landing control of a spacecraft would be preferable to manual control. The corresponding classification task involves distinguishing among different operational conditions based on a set of numerical measurements [61].
19. **Spambase**. The Spambase dataset contains features extracted from email messages, including word and character frequencies as well as information related to sequences of capital letters. The corresponding binary classification task aims to distinguish spam from non-spam emails [62].
20. **Spiral**, The Spiral dataset is a synthetic binary classification problem consisting of 1,000 two-dimensional instances equally distributed between two classes. The instances are generated using parametric equations that form two intertwined spiral patterns, resulting in a highly nonlinear classification problem [63].
21. **Student.** The Student dataset contains educational data collected from 587 students over two consecutive academic years in ten non-university educational centers. The attributes describe factors such as academic performance, student abilities, motivation, and methodological and assessment preferences. The corresponding classification task aims to categorize students according to their educational characteristics and performance [64].
22. **Splice** dataset, which is a biological classification benchmark consisting of 3,190 DNA sequences, each represented by 60 categorical features. The task is to classify sequences into three categories: exon-intron boundaries (EI), intron-exon boundaries (IE), and non-splice junctions (N) [65] .
23. **Texture.** The Texture dataset is designed for the classification of different visual textures and comprises 11 texture classes, including grass lawn, pressed calf leather, handmade paper, raffia, and cotton canvas. Each pattern corresponds to a pixel represented by 40 numerical features obtained from the estimation of fourth-order modified moments computed at four different orientations: $0^\circ, 45^\circ, 90^\circ$, and 135°. The corresponding classification task aims to assign each pattern to one of the 11 texture categories [66].
24. **Thyroid.** The Thyroid dataset contains clinical measurements related to thyroid gland function. The corresponding classification task aims to determine whether a patient exhibits normal thyroid function or suffers from hyperthyroidism or hypothyroidism, resulting in a three-class classification problem [67].

25. The **TwoNorm** dataset is a synthetic 20-dimensional binary classification problem originally introduced by Breiman. The patterns of each class are generated from multivariate normal distributions with unit covariance matrices and symmetric mean vectors. Specifically, the two class means are $(a, a, \ldots, a)$ and $(-a, -a, \ldots, -a)$, where $a = 2/\sqrt{20}$ [68].
26. **Vehicle.** The Vehicle dataset is designed to classify vehicle silhouettes into four different categories using a set of numerical features extracted from their shapes. The silhouettes were obtained from vehicles viewed from different angles, making the classification task dependent on distinguishing characteristic geometric properties despite variations in viewing orientation [69].
27. **Yeast.** The Yeast dataset contains information describing proteins from Saccharomyces cerevisiae. Each protein is represented by eight numerical features derived from sequence-based measurements. The corresponding classification task aims to predict the cellular localization site of each protein among ten possible classes [70].
28. **EEG**. The EEG dataset is a medical dataset containing electroencephalographic recordings obtained from healthy subjects and patients with epilepsy under different physiological and pathological conditions [71,72]. The original dataset consists of five subsets, denoted by Z, O, N, F, and S, corresponding to different recording conditions and brain states. From these subsets, four classification problems were constructed for the experimental evaluation: Z_F_S, ZO_NF_S, Z_O_N_F_S, and ZONF_S.

Table 1. The main characteristics of each dataset.

DATASET	PATTERNS	CLASSES
ALCOHOL	476	4
APPENDICITIS	106	2
BEED	8000	4
DERMATOLOGY	358	6
GENE	3175	3
HAYES ROTH	160	3
HEPATITIS	80	2
HOUSEVOTES	232	2
IONOSPHERE	351	2
LYMPHOGRAPHY	148	4
NURSERY	12960	5
OPTDIGITS	5620	10
PENBASED	10992	10
PHONEME	5404	2
PIRVISION	15302	3
SATIMAGE	6435	7
SEGMENT	2310	7
SHUTTLE	58000	7
SPAMBASE	4597	2
SPIRAL	2000	2
SPLICE	3190	3
STUDENT	587	4
TEXTURE	5500	11
THYROID	7200	3
TWONORM	7400	2
VEHICLE	846	4
YEAST	1484	10
Z_F_S	300	3
Z_O_N_F_S	500	5
ZO_NF_S	500	3
ZONF_S	500	2

3.2. Experimental results

The proposed method was implemented in C++. To ensure the reliability of the experimental evaluation, each experiment was independently repeated 30 times, using a different random seed in each run. The code used in the experiments with GenClass is a modifed version of the original algorithm located in the repository https://github.com/itsoulos/QGenClass/(accessed on 24 October 2026). Furthermore, 10-fold cross-validation was employed to assess the generalization performance of the method and provide a robust evaluation of the experimental results. Values in tables represent average test error as measured on the corresponding test set. The values used for the experimental parameters are listed in Table 2.

Table 2. Values for the parameters in experiments.

PARAMETER	MEANING	VALUE
N_c	Number of chromosomes	500
N_g	Number of allowed generations	500
p_s	Selection rate	0.9
p_m	Mutation rate	0.05
N_K	Crossover items	10
k	Number of crossover trials	20
ρ	Elite fraction for donor selection	0.20
$B_{\max}$	Maximum crossover block size	16

The notation used in Table 4 is described below:

1. DATASET denotes the dataset used in the corresponding experiment.
2. RBF denotes a Radial Basis Function (RBF) neural network [73,74] with $H = 10$ hidden nodes.
3. NEAT denotes the NeuroEvolution of Augmenting Topologies (NEAT) method [75].
4. DNN denotes a deep neural network implemented using the Tiny-DNN library, publicly available at the project's GitHub repository (accessed on 3 October 2026). The network was trained using the AdaGrad optimization algorithm [76].
5. ADAM denotes a neural network with $H = 10$ hidden nodes trained using the Adam optimization algorithm [77].
6. BFGS denotes a neural network with $H = 10$ hidden nodes trained using Powell's variant of the BFGS optimization method [78].
7. GEN denotes a neural network with $H = 10$ hidden nodes whose weights were optimized using a Genetic Algorithm. The Genetic Algorithm employed the same parameter settings reported in Table 2.
8. CLASS denotes the original GenClass rule construction method without the additional crossover mechanism proposed in this work.
9. STANDARD denotes GenClass enhanced with the proposed Standard crossover method.
10. The row AVERAGE reports the mean classification error across all datasets for the corresponding method.

Figure 4 compares the average classification errors of the evaluated methods with the majority-class baseline. The STANDARD method achieves the lowest average classification error (13.42%), followed by TARGETED (15.08%), TARGETEDWORST (15.42%), and the original CLASS method (16.13%). In contrast, the majority-class baseline yields an average error of 56.23%. Although some competing classifiers perform worse than the majority-class baseline on individual datasets, the four GenClass variants consistently achieve lower classification errors across all 31 datasets.

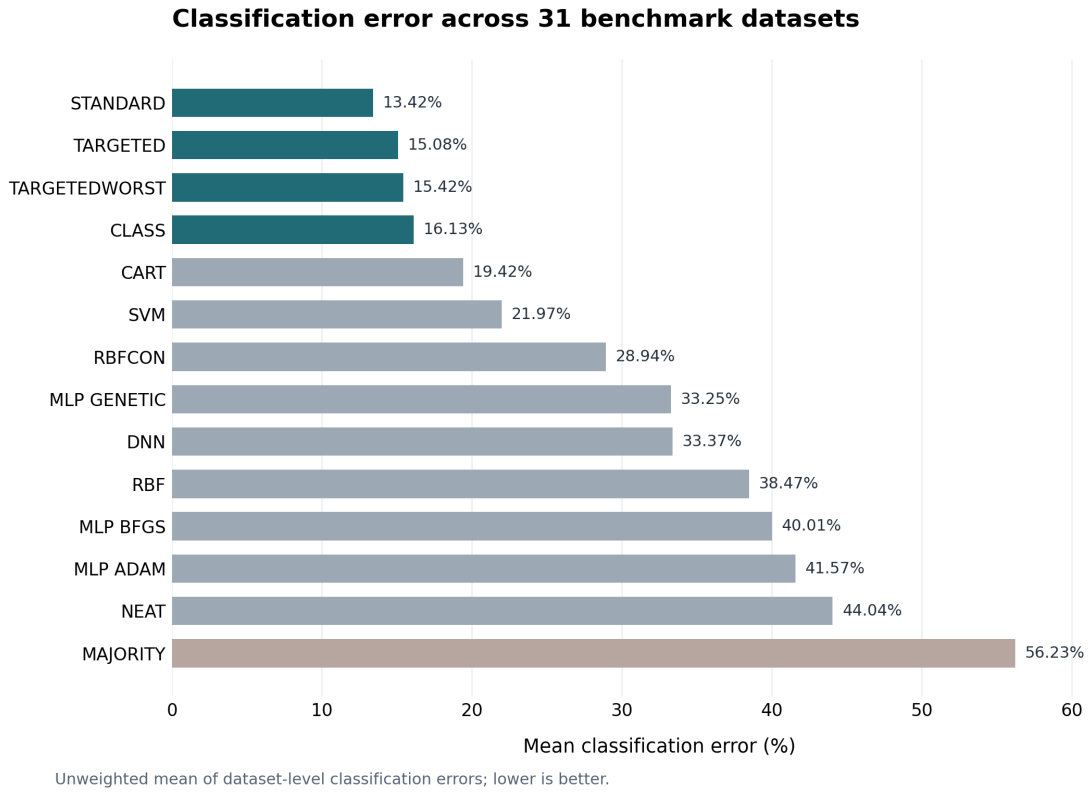


Figure 4. Average classification error (%) of the evaluated classification methods across 31 benchmark datasets. The reported values represent the unweighted mean of the dataset-level classification errors. Lower values indicate better classification performance. The majority-class classifier is included as a simple reference baseline.

To assess the influence of the additional computational effort introduced by the crossover mechanism, we performed a supplementary comparison between STANDARD and the original CLASS method under equal fitness evaluation budgets. For each corresponding experimental run, the total number of fitness evaluations performed by STANDARD was recorded and subsequently used as the maximum evaluation budget for CLASS.

The results are presented in Table 3. STANDARD achieved an average classification error of 13.71%, compared with 14.21% for CLASS. Although STANDARD retained a small numerical advantage, the difference was not statistically significant according to the Wilcoxon signed-rank test ($p \approx 0.608$).

These findings suggest that the additional computational effort associated with the crossover mechanism may contribute to the improvements observed in the original generation-based comparison. Nevertheless, the proposed method maintains a slightly lower average classification error when both approaches are evaluated under equal fitness evaluation budgets.

Statistical analysis. The comparison of the methods was based on the average classification error of each method on each dataset, so that every dataset provides one observation and $N = 31$ observations are available for each method. All tests were performed at the significance level $\alpha = 0.05$. For every table, the Friedman test was first applied in order to examine the hypothesis that all the methods of the table perform equally. For each dataset the methods are ranked from the lowest to the highest error, with ties receiving the average rank, and the test compares the mean ranks of the methods. The result is also reported with the Iman-Davenport correction, which is less conservative, and the strength of the differences is expressed by Kendall's coefficient of concordance $W = \chi^2/(N(k-1))$, where k is the number of methods. The Nemenyi test was then used to compare all the

Table 3. Classification error (%) of STANDARD and CLASS under equal fitness evaluation budgets.

Dataset	STANDARD	CLASS-EQUAL
Alcohol	**10.43**	12.13
Appendicitis	15.00	**14.00**
Beed	**25.68**	27.78
Dermatology	6.00	**5.14**
Gene	7.35	**6.85**
Hayes-Roth	27.69	**22.31**
Hepatitis	**13.75**	17.50
Housevotes	**3.48**	3.89
Ionosphere	10.00	**7.43**
Lymography	24.29	**22.86**
Nursery	19.10	**18.41**
Optdigits	**13.29**	16.94
Penbased	**14.52**	21.68
Phoneme	**16.73**	17.54
Pir	1.40	**1.19**
Satimage	15.26	**15.23**
Segment	**9.05**	10.35
Shuttle	1.05	**0.20**
Spambase	**6.93**	7.76
Spiral	**28.90**	34.20
Splice	7.21	**6.90**
Student	7.45	**5.73**
Texture	**8.36**	12.91
Thyroid	**0.64**	0.96
Twonorm	5.57	**2.82**
Vehicle	**35.83**	36.07
Wine	8.82	8.82
Yeast	46.83	**45.15**
Z_F_S	**6.33**	6.67
Z_O_N_F_S	**25.40**	28.80
ZONF_S	2.80	**2.20**
Mean	**13.71**	14.21

methods through their mean ranks, and two methods are considered different when their mean ranks differ by more than the critical difference CD. In addition, specific pairs of methods were compared with the two-sided Wilcoxon signed-rank test for paired samples, applied to the differences of the errors on the individual datasets, where zero differences are discarded and tied absolute differences receive average ranks. The p-values of the Wilcoxon tests were adjusted with the Holm method separately for each table, over the family of comparisons of that table. These are the seven comparisons of STANDARD with each of the other methods in Table 4, and all the pairs of methods in Table 5 (three comparisons) and in Table 6 (six comparisons). Finally, for Table 6, the one-sided Page trend test was used to examine whether the error decreases as N_K increases, together with the Spearman correlation between N_K and the error in each dataset, whose sign was summarized with a sign test.

Table 4 compares the proposed method (STANDARD) with the original GenClass method (CLASS) and with the other classifiers over the 31 datasets. All values are classification errors, so lower values indicate better performance. STANDARD attained the lowest average error, 13.42%, followed by CLASS with 16.13%, while the average errors of the remaining methods ranged from 33.25% (GEN) to 44.04% (NEAT). STANDARD achieved the lowest error in 23 of the 31 datasets, CLASS in 3, and each of the other methods in at most 2. Compared with CLASS, the average error decreases by 2.71 percentage points, a relative reduction of 16.8%. Therefore, the new crossover procedure improves the original method on average, while the improvement over the other methods is considerably larger.

The critical difference of the Nemenyi test is $CD = \frac{q_\alpha}{\sqrt{2}}\sqrt{\frac{k(k+1)}{6N}}$, where q_α is the critical value of the studentized range distribution for k methods, and it therefore depends on the number of methods compared. For each pair of methods, the size of the difference was estimated with the Hodges-Lehmann estimator, which is the median of the pairwise averages of the paired differences of the errors, and its 95% confidence interval was derived from the Wilcoxon signed-rank test using the normal approximation. The effect size of each Wilcoxon test was expressed as $r = Z/\sqrt{n}$, where Z is the standardized statistic of the test and n is the number of datasets with non-zero difference. The confidence intervals of the mean rank and of the mean change in error in Figure 7 were computed with the percentile bootstrap, using 10,000 resamples of the datasets with replacement.

Notation for Figures 5, 6 and 7. Each figure has four panels and a header with the Friedman test, the Iman-Davenport correction and Kendall's W.

- *Box plots (top left): The box is the interquartile range (IQR), the thick line is the median, the whiskers extend up to* $1.5 \times IQR$*, the diamond is the mean, the dots are the datasets and the grey lines join the same dataset.*
- *Pairs "A vs B" (top right): A is the first method named and B the second. The dot is the Hodges-Lehmann median paired difference, error(B) – error(A), with its 95% confidence interval. Right of the dashed line, A has lower error. The label gives the Holm-adjusted Wilcoxon p-value and the wins/ties/losses of A.*
- *Mean ranks (bottom left): Mean Friedman rank (1 = best)* $\pm CD/2$*, where CD is the Nemenyi critical difference. Non-overlapping bars differ at* $\alpha = 0.05$*.*
- *Bottom right: In Figures 5 and 6, the per-dataset differences error(B) – error(A) (blue: A better, red: B better). In Figure 7, the mean change in error relative to* $N_K = 2$ *and the mean rank as a function of* N_K*, with bootstrap 95% confidence intervals.*
- *Stars: *** $p<0.001$, ** $p<0.01$, * $p<0.05$, ns = not significant (Holm-adjusted).*

Table 4. Classification error (%) of the proposed method (STANDARD) and of the other classifiers on the 31 datasets. STANDARD is the GenClass method with the proposed additional crossover procedure, while CLASS is the original GenClass method without it. The last row (AVERAGE) reports the mean error over all datasets, and lower values indicate better performance.

DATASET	RBF	NEAT	DNN	ADAM	BFGS	GEN	CLASS	STANDARD
ALCOHOL	49.38%	66.80%	39.04%	57.78%	41.50%	39.57%	16.70%	11.70%
APPENDICITIS	12.23%	17.20%	17.30%	16.50%	18.00%	18.10%	16.47%	12.80%
BEED	65.12%	67.94%	64.45%	70.38%	75.24%	46.19%	31.93%	27.45%
DERMATOLOGY	59.48%	32.43%	24.26%	26.14%	52.92%	30.58%	5.69%	2.80%
GENE	37.11%	52.68%	33.06%	35.17%	38.12%	34.38%	8.67%	7.29%
HAYES ROTH	64.36%	50.15%	44.65%	59.70%	37.33%	56.18%	24.31%	21.08%
HEPATITIS	18.38%	39.75%	16.65%	16.25%	21.38%	15.00%	15.50%	12.00%
HOUSEVOTES	6.13%	10.89%	3.13%	7.48%	7.13%	6.62%	3.47%	2.96%
IONOSPHERE	16.22%	19.67%	12.57%	16.64%	15.29%	15.14%	8.10%	8.74%
LYMPHOGRAPHY	25.50%	33.70%	24.07%	39.79%	35.43%	28.42%	22.67%	18.86%
NURSERY	77.10%	64.11%	59.73%	40.94%	41.40%	32.14%	19.65%	22.27%
OPTDIGITS	71.97%	72.56%	64.34%	72.32%	67.13%	67.81%	27.15%	14.77%
PENBASED	81.69%	78.58%	73.50%	77.41%	82.48%	84.01%	24.93%	17.79%
PHONEME	23.32%	22.43%	24.34%	29.43%	15.58%	15.55%	18.89%	16.82%
PIRVISION	24.58%	11.84%	18.28%	18.38%	16.94%	14.31%	6.53%	4.38%
SATIMAGE	40.19%	76.19%	72.30%	73.22%	71.73%	54.54%	18.47%	16.14%
SEGMENT	59.68%	66.72%	32.04%	49.75%	68.97%	57.72%	15.21%	9.57%
SHUTTLE	32.83%	29.32%	2.27%	21.29%	42.07%	20.86%	1.11%	1.28%
SPAMBASE	29.35%	35.77%	11.30%	48.05%	18.16%	6.37%	8.97%	7.28%
SPIRAL	44.87%	48.66%	45.24%	47.67%	47.99%	48.66%	35.65%	31.25%
SPLICE	40.27%	55.54%	48.18%	75.93%	30.64%	30.97%	7.68%	7.62%
STUDENT	5.49%	10.20%	6.93%	5.13%	7.14%	5.61%	6.86%	5.72%
TEXTURE	74.52%	70.82%	70.78%	56.33%	59.25%	54.00%	20.01%	10.71%
THYROID	10.62%	18.06%	7.42%	3.43%	5.18%	7.65%	1.24%	0.83%
TWONORM	2.18%	13.54%	2.16%	5.58%	9.22%	9.59%	6.77%	6.57%
VEHICLE	78.06%	64.05%	59.22%	74.41%	72.99%	60.24%	36.55%	35.83%
YEAST	67.14%	71.09%	68.87%	57.69%	77.28%	69.13%	45.95%	44.35%
Z_F_S	13.16%	38.41%	9.27%	47.81%	39.37%	10.73%	7.39%	6.00%
Z_O_N_F_S	48.70%	77.08%	67.80%	78.79%	65.67%	64.81%	30.32%	25.32%
ZO_NF_S	9.02%	43.75%	8.50%	47.43%	43.04%	21.54%	5.03%	3.69%
ZONF_S	4.03%	5.44%	2.52%	11.99%	15.62%	4.36%	2.04%	2.00%
AVERAGE	**38.47%**	**44.04%**	**33.37%**	**41.57%**	**40.01%**	**33.25%**	**16.13%**	**13.42%**

The statistical analysis of Table 4 is summarized in Figure 5. The Friedman test rejects the hypothesis that all eight methods perform equally ($\chi^2(7) = 121.47$, $p = 3.8 \times 10^{-23}$), a result confirmed by the Iman-Davenport correction ($F(7, 210) = 38.15$, $p = 3.3 \times 10^{-34}$), with a substantial effect size (Kendall's $W = 0.56$). STANDARD obtains the best mean rank (1.42), followed by CLASS (2.29), while all the other methods have mean ranks above 4. In the paired comparisons, STANDARD has lower error than every other method, with Holm-adjusted Wilcoxon signed-rank p-values between 8.6×10^{-6} and 1.7×10^{-5} and wins in 28 to 31 of the 31 datasets. The Hodges-Lehmann estimates of the median paired difference range from approximately 2.4 percentage points (CLASS) to 30.4 percentage points (NEAT), none of the 95% confidence intervals includes zero, and the effect sizes are large ($r \geq 0.77$, where values around 0.5 or above are conventionally considered large). The comparison with CLASS requires a careful reading. The bars of the two methods overlap in the mean-rank panel, which means that the conservative Nemenyi test does not separate them, since the difference in mean rank (0.87) is smaller than the critical difference (1.89). The paired Wilcoxon test, which also takes into account the magnitude of the differences, detects nevertheless a consistent improvement (28 wins and 3 losses). STANDARD is worse than CLASS only in three datasets (IONOSPHERE, NURSERY and SHUTTLE).

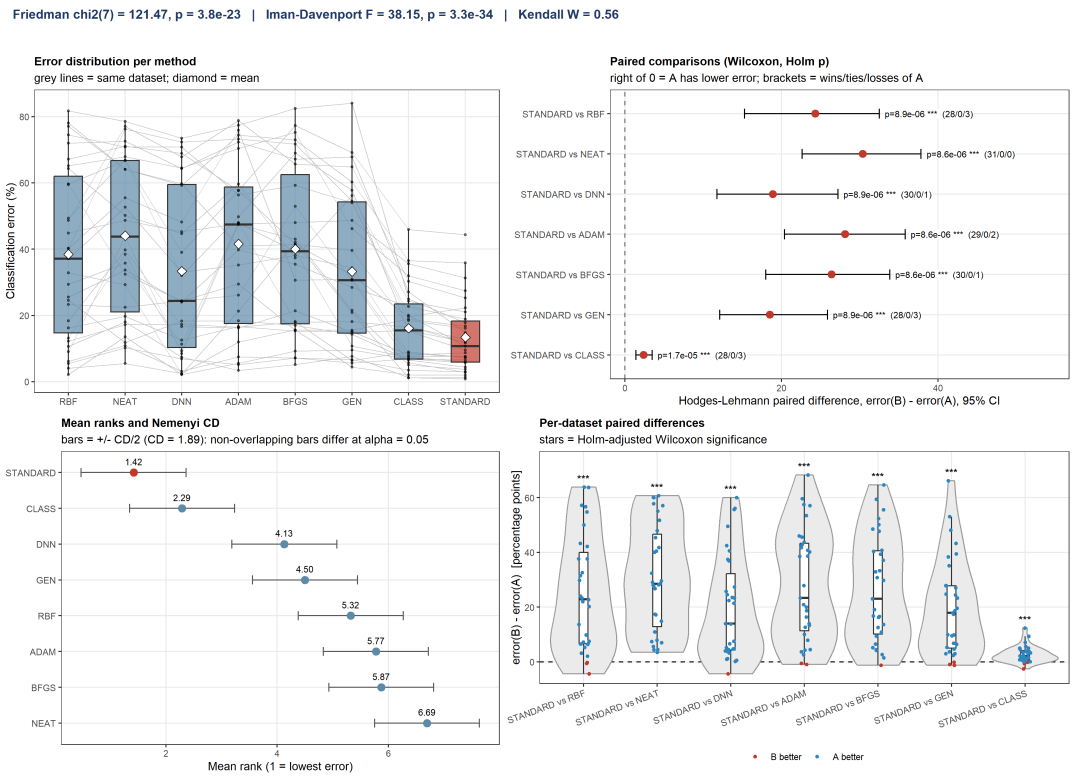


Figure 5. Statistical comparison of the proposed method (STANDARD) against the other classifiers over 31 datasets.

3.3. Crossover methods

Table 5 isolates the effect of the crossover strategy. STANDARD, in which the exchanged chromosome regions are selected purely stochastically, achieves the lowest average error (13.42%), whereas the two strategies that exploit information collected during the evolutionary process are less effective, with 15.08% for TARGETED and 15.42% for TARGETEDWORST. STANDARD obtained the lowest error in 24 of the 31 datasets, compared with 5 for TARGETED and 3 for TARGETEDWORST (a tie between the two targeted strategies in IONOSPHERE is counted for both). The average gain of STANDARD is 1.66 percentage points over TARGETED and 2.0 percentage points over TARGETEDWORST, which correspond to relative reductions of 11.0% and 13.0%, respectively. The two targeted strategies differ little from each other. Hence, in these experiments, guiding the exchange of genetic material with information from the evolutionary process did not improve upon the purely random exchange.

Table 5. Classification error (%) of the three crossover strategies.

DATASET	STANDARD	TARGETED	TARGETEDWORST
ALCOHOL	11.70%	17.65%	14.04%
APPENDICITIS	12.80%	16.40%	15.40%
BEED	27.45%	29.48%	31.93%
DERMATOLOGY	2.80%	6.12%	5.14%
GENE	7.29%	9.05%	8.20%
HAYES ROTH	21.08%	22.00%	25.38%
HEPATITIS	12.00%	16.25%	16.25%
HOUSEVOTES	2.96%	3.72%	3.04%
IONOSPHERE	8.74%	7.77%	7.77%
LYMPHOGRAPHY	18.86%	25.00%	21.86%
NURSERY	22.27%	21.44%	21.02%
OPTDIGITS	14.77%	19.59%	19.88%
PENBASED	17.79%	15.10%	23.85%
PHONEME	16.82%	17.86%	17.86%
PIRVISION	4.38%	4.47%	6.53%
SATIMAGE	16.14%	17.52%	18.14%
SEGMENT	9.57%	12.86%	11.26%
SHUTTLE	1.28%	1.35%	0.94%
SPAMBASE	7.28%	8.22%	8.10%
SPIRAL	31.25%	35.30%	34.83%
SPLICE	7.62%	8.12%	8.15%
STUDENT	5.72%	5.92%	6.45%
TEXTURE	10.71%	17.23%	16.59%
THYROID	0.83%	1.35%	1.53%
TWONORM	6.57%	3.90%	5.00%
VEHICLE	35.83%	39.64%	37.03%
YEAST	44.35%	45.22%	47.77%
Z_F_S	6.00%	6.40%	7.33%
Z_O_N_F_S	25.32%	28.12%	30.66%
ZO_NF_S	3.69%	3.08%	4.04%
ZONF_S	2.00%	1.28%	2.04%
AVERAGE	**13.42%**	**15.08%**	**15.42%**

Figure 5 presents the statistical analysis of Table 5. The Friedman test indicates significant differences among the three strategies ($\chi^2(2) = 22.13$, $p = 1.6 \times 10^{-5}$, Iman-Davenport $F(2, 60) = 16.65$, $p = 1.8 \times 10^{-6}$), with a moderate effect size ($W = 0.36$). STANDARD has the best mean rank (1.32), against 2.27 for TARGETED and 2.40 for TARGETEDWORST. Since the critical difference is 0.60, the bar of STANDARD does not overlap with those of the other two strategies, while the bars of TARGETED and TARGETEDWORST overlap. The paired tests agree with this picture. STANDARD is significantly better than TARGETED (25 wins and 6 losses, median difference 1.62 percentage points, 95% CI $[0.58, 2.55]$, $p = 0.0018$, $r = 0.60$) and than TARGETEDWORST (27 wins and 4 losses, median difference 1.93 percentage points, 95% CI $[1.08, 2.80]$, $p = 1.6 \times 10^{-4}$, $r = 0.73$). On the contrary, TARGETED and TARGETEDWORST cannot be distinguished (15 wins, 3 ties and 13 losses, median difference approximately 0.1 percentage points, 95% CI $[-0.48, 0.95]$, $p = 0.64$, $r = 0.09$).

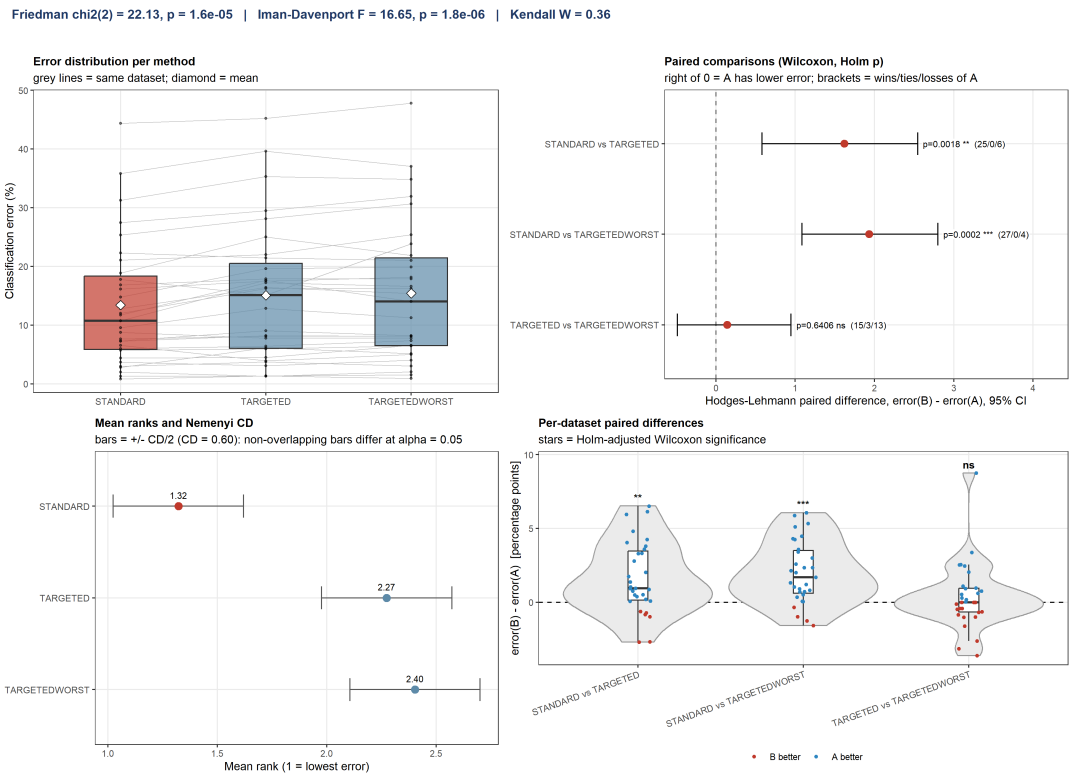


Figure 6. Statistical comparison of the three crossover strategies. (STANDARD, TARGETED, TARGETEDWORST).

3.4. Number of crossing items

Table 6 reports the results of the proposed method when N_K=2, 5, 10 and 15 chromosomes participate in the crossover. The column N_K=10 corresponds to the default value of Table 2 and coincides with the STANDARD column of Table 5. The average error decreases monotonically as N_K increases, from 14.58% for N_K=2 to 13.66% for N_K=5, 13.42% for N_K=10 and 13.19% for N_K=15. The setting N_K=15 gave the lowest error in 17 of the 31 datasets, N_K=5 and N_K=10 in 7 datasets each, and N_K=2 in none. Increasing N_K from 2 to 15 reduces the average error by 1.39 percentage points, whereas each of the two further increases beyond N_K=5 changes it by less than 0.3 percentage points.

Table 6. Experiments with the number that participate in the crossover procedure.

DATASET	$N_K = 2$	$N_K = 5$	$N_K = 10$	$N_K = 15$
ALCOHOL	12.55%	13.19%	11.70%	15.11%
APPENDICITIS	15.60%	14.40%	12.80%	15.80%
BEED	27.30%	26.78%	27.45%	25.47%
DERMATOLOGY	4.74%	4.86%	2.80%	4.00%
GENE	8.39%	7.29%	7.29%	6.91%
HAYES ROTH	20.77%	19.54%	21.08%	21.39%
HEPATITIS	14.00%	11.50%	12.00%	10.00%
HOUSEVOTES	3.63%	2.78%	2.96%	3.28%
IONOSPHERE	7.60%	7.31%	8.74%	8.80%
LYMPHOGRAPHY	23.71%	23.00%	18.86%	20.57%
NURSERY	20.33%	22.16%	22.27%	18.76%
OPTDIGITS	17.16%	14.56%	14.77%	13.99%
PENBASED	19.95%	16.54%	17.79%	14.22%
PHONEME	16.99%	16.21%	16.82%	16.00%
PIRVISION	8.49%	6.33%	4.38%	5.35%
SATIMAGE	15.91%	15.04%	16.14%	14.93%
SEGMENT	11.95%	9.91%	9.57%	9.44%
SHUTTLE	1.15%	0.62%	1.28%	0.24%
SPAMBASE	7.70%	7.98%	7.28%	7.33%
SPIRAL	36.80%	33.55%	31.25%	30.35%
SPLICE	7.46%	7.18%	7.62%	7.34%
STUDENT	6.11%	4.92%	5.72%	6.70%
TEXTURE	12.87%	12.73%	10.71%	12.44%
THYROID	1.57%	1.06%	0.83%	0.76%
TWONORM	5.62%	5.31%	6.57%	8.01%
VEHICLE	37.62%	34.81%	35.83%	33.69%
YEAST	46.21%	45.69%	44.35%	44.01%
Z_F_S	7.00%	6.47%	6.00%	5.67%
Z_O_N_F_S	27.28%	27.00%	25.32%	23.80%
ZO_NF_S	3.68%	3.32%	3.69%	3.40%
ZONF_S	1.96%	1.44%	2.00%	1.16%
AVERAGE	**14.58%**	**13.66%**	**13.42%**	**13.19%**

Figure 7 presents the corresponding statistical analysis. The Friedman test detects significant differences among the four settings ($\chi^2(3) = 21.97$, $p = 6.6 \times 10^{-5}$, Iman-Davenport $F(3, 90) = 9.28$, $p = 2.1 \times 10^{-5}$), although with a smaller effect size than in the previous comparisons ($W = 0.24$). The mean rank is the worst for $N_K = 2$ (3.35) and the best for $N_K = 15$ (1.87), with $N_K = 5$ (2.27) and $N_K = 10$ (2.50) in between. The one-sided Page trend test, which examines whether the error decreases as N_K increases, confirms a significant trend ($z = 4.08$, $p = 2.3 \times 10^{-5}$). In agreement, the Spearman correlation between N_K and the error is negative in 23 of the 29 datasets where it is non-zero (median correlation -0.6, sign test $p = 0.0023$). The pairwise comparisons show, however, that this trend is mainly due to the weakness of the smallest setting. The value $N_K = 2$ has significantly higher error than $N_K = 5$ ($p = 3.8 \times 10^{-4}$), $N_K = 10$ ($p = 0.0047$) and $N_K = 15$ ($p = 0.0039$), with median differences of approximately 0.8, 1.0 and 1.3 percentage points, respectively. None of the differences among $N_K = 5$, 10 and 15 is statistically significant (adjusted $p = 0.61$ for 5 versus 10, 0.20 for 5 versus 15 and 0.61 for 10 versus 15), and their confidence intervals include zero. Therefore, a very small number of chromosomes in the crossover is detrimental, whereas beyond $N_K = 5$ any further benefit is small and is not statistically established in these experiments, although $N_K = 15$ has the best mean rank.

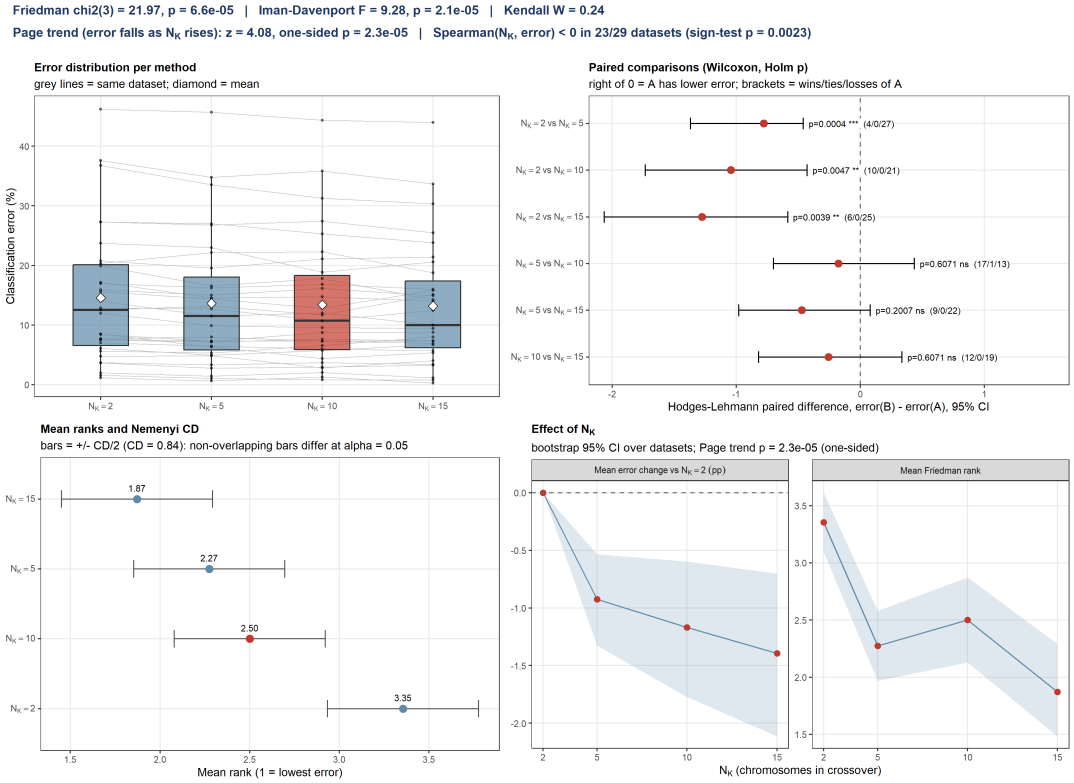


Figure 7. Effect of the number of chromosomes (N_K) participating in the crossover on the proposed method.

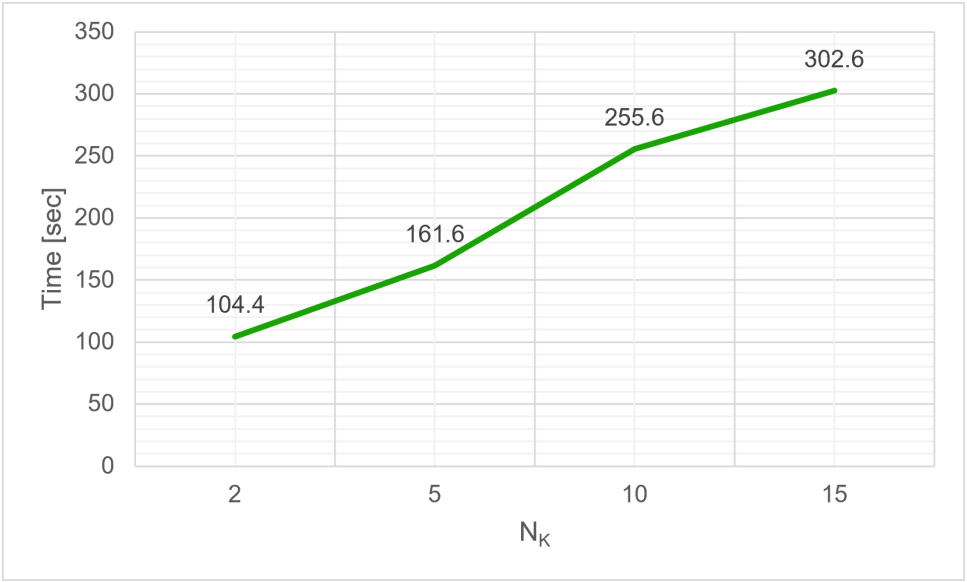


Figure 8. Execution time of the Standard technique for different values of N_K (Dermatology dataset).

The figure shows the average execution time of the Standard technique on the Dermatology dataset for N_K=2, 5, 10 and 15. The time increases withN_K, from 104.4 s for $N_K = 2$ to 302.6 s for $N_K = 15$, which is about 2.9 times higher. The increase is approximately linear up to $N_K = 10$, at about 19 s per additional chromosome, and becomes slower from $N_K = 10$ to $N_K = 15$, at about 9 s per additional chromosome. The time does not grow in proportion to N_K, because the rest of the algorithm has a cost that does not depend on N_K. The default value $N_K = 10$ requires 255.6 s, whereas $N_K = 15$ needs 47 s more (18%) and, in this dataset, does not give a lower error (Table 6, 4.00% against 2.80% for $N_K = 10$).

4. Conclusions

This work introduced an additional crossover procedure into the GenClass method. At every generation, after the standard genetic operators, N_K randomly selected chromosomes undergo k crossover trials, and a candidate chromosome replaces the current one only if it achieves a lower classification error on the training set. The procedure was examined in three variants (Standard, Targeted and TargetedWorst) on 31 classification datasets. The Standard variant combines each selected chromosome with randomly chosen chromosomes of the population using one-point crossover. It reduced the average classification error of the original method from 16.13% to 13.42%, a relative reduction of 16.8%, and obtained the lowest error in 23 of the 31 datasets. According to the paired Wilcoxon test with Holm correction, the improvement was statistically significant, and the margin over all the other classifiers of the comparison was considerably larger.

The comparison of the three variants gave a result that differs from what their design might suggest. The Targeted and TargetedWorst variants take donor chromosomes only from the best fraction of the population and transfer a bounded block of codons between semantically compatible positions, yet they did not improve on the random exchange of Standard. Standard was significantly better than both, while the two targeted variants could not be distinguished from each other, so the additional focus of TargetedWorst on the worst-performing class brought no measurable benefit over Targeted. All three variants obtained a lower average error than the original method (13.42%, 15.08% and 15.42%, against 16.13%), but statistical significance against the original method was examined only for Standard. A possible explanation is related to the difference in the exchanged material. In Standard the donors come from the whole population and the genetic material after a random crossover point is exchanged, whereas in the targeted variants the donors come from the best 20% of the population and the exchanged block is limited to at most $B_{\max}$ codons. The unrestricted exchange may therefore introduce more diverse genetic material, which relates to the loss of diversity identified as a limitation of the original method. This hypothesis was not tested directly in the present work.

The number of chromosomes participating in the crossover affects the performance, mainly at its lowest values. A very small number ($N_K = 2$) was significantly worse than the other settings, whereas the differences among $N_K = 5$, 10 and 15 were small and not statistically significant, although $N_K = 15$ obtained the lowest average error and the best mean rank. A moderate value of N_K therefore seems sufficient.

Some limitations should be noted. The additional procedure requires up to $N_K k$ extra fitness evaluations per generation, that is 200 with the parameter values of Table 2, while the standard genetic operators generate $p_s N_c = 450$ offspring per generation. Moreover, the parameters k, ρ and $B_{\max}$ were kept fixed and their influence was not examined. The conclusions are limited to the datasets, the parameter values and the classification task used in the experiments.

Several directions for future work arise from the present study. First, the reasons why the targeted variants did not outperform the random one should be investigated, for example by measuring the diversity of the population during the evolution, by examining the influence of the elite fraction ρ and of the maximum block size $B_{\max}$, and by designing guided operators that preserve diversity or combine random and guided exchanges. Second, the number of chromosomes N_K and the number of trials k could be adapted automatically during the evolution instead of being fixed, and values of N_K larger than 15 could be examined, since $N_K = 15$ had the best mean rank without a statistically established advantage. Third, the computational cost of the procedure should be measured and analyzed together with possible parallel implementations. Finally, the procedure could

be evaluated on larger datasets, on other learning tasks such as regression, and in other methods based on Grammatical Evolution.

Author Contributions: G.G. and I.G.T. conducted the experiments, employing several datasets and provided the comparative experiments. V.C. performed the statistical analysis and prepared the manuscript. All authors have read and agreed to the published version of the manuscript.

Funding: This research has been financed by the European Union : Next Generation EU through the Program Greece 2.0 National Recovery and Resilience Plan , under the call RESEARCH – CREATE – INNOVATE, project name “iCREW: Intelligent small craft simulator for advanced crew training using Virtual Reality techniques" (project code:TAEDK-06195).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. J.H. Holland, Genetic algorithms. Scientific american **267**, pp. 66-73, 1992.
2. N. Yusup, A. M. Zain, S. Z. M. Hashim, Evolutionary techniques in optimizing machining parameters: Review and recent applications (2007–2011), Expert Systems with Applications **39.10**, pp. 9909-9927, 2012.
3. J. Stender, Parallel Genetic Algorithms:Theory & Applications. Edition: IOS Press, 1993.
4. D. Goldberg, Genetic Algorithms in Search, Optimization and Machine Learning, Addison-Wesley Publishing Company, Reading, Massachussets, 1989.
5. Z. Michaelewicz, Genetic Algorithms + Data Structures = Evolution Programs. Springer - Verlag, Berlin, 1996.
6. Y.H. Santana, R.M. Alonso, G.G. Nieto, L. Martens, W. Joseph, D. Plets, Indoor genetic algorithm-based 5G network planning using a machine learning model for path loss estimation, Appl. Sci. **12**, 3923. 2022.
7. X. Liu, D. Jiang, B. Tao, G. Jiang, Y. Sun, J. Kong, B. Chen, Genetic algorithm-based trajectory optimization for digital twin robots, Front. Bioeng. Biotechnol **9**, 793782, 2022.
8. K. Nonoyama, Z.Liu, T. Fujiwara, M.M. Alam, T. Nishi, Energy-efficient robot configuration and motion planning using genetic algorithm and particle swarm optimization, Energies **15**, 2074, 2022.
9. K. Liu, B. Deng, Q. Shen, J. Yang, Y. Li, Optimization based on genetic algorithms on energy conservation potential of a high speed SI engine fueled with butanol–gasoline blends, Energy Rep. **8**, pp. 69–80, 2022.
10. G. Zhou, S. Zhu, S. Luo, Location optimization of electric vehicle charging stations: Based on cost model and genetic algorithm, Energy **247**, 123437, 2022.
11. Choudhury, S.; Rana, M.; Chakraborty, A.; Majumder, S.; Roy, S.; RoyChowdhury, A.; Datta, S. Design of patient specific basal dental implant using Finite Element method and Artificial Neural Network technique. J. Eng. Med. 2022, 236, 1375–1387.
12. Chen, Q.; Hu, X. Design of intelligent control system for agricultural greenhouses based on adaptive improved genetic algorithm for multi-energy supply system. Energy Rep. 2022, 8, 12126–12138.
13. M. O’Neill, C. Ryan, Grammatical evolution, IEEE Trans. Evol. Comput. **5,**pp. 349–358, 2001.
14. J. W. Backus. The Syntax and Semantics of the Proposed International Algebraic Language of the Zurich ACM-GAMM Conference. Proceedings of the International Conference on Information Processing, UNESCO, 1959, pp.125-132.
15. C. Ryan, J. Collins, M. O’Neill, Grammatical evolution: Evolving programs for an arbitrary language. In: Banzhaf, W., Poli, R., Schoenauer, M., Fogarty, T.C. (eds) Genetic Programming. EuroGP 1998. Lecture Notes in Computer Science, vol 1391. Springer, Berlin, Heidelberg, 1998.
16. M. O’Neill, M., C. Ryan, Evolving Multi-line Compilable C Programs. In: Poli, R., Nordin, P., Langdon, W.B., Fogarty, T.C. (eds) Genetic Programming. EuroGP 1999. Lecture Notes in Computer Science, vol 1598. Springer, Berlin, Heidelberg, 1999.
17. A. Brabazon, M. O’Neill, Credit classification using grammatical evolution, Informatica **30.3**, 2006.
18. S. Şen, J.A. Clark. A grammatical evolution approach to intrusion detection on mobile ad hoc networks, In: Proceedings of the second ACM conference on Wireless network security, 2009.
19. I.G. Tsoulos, I.E. Lagaris, Solving differential equations with genetic programming, Genet Program Evolvable Mach **7**, pp. 33–54, 2006.

20. L. Chen, C.H. Tan, S.J. Kao, T.S. Wang, Improvement of remote monitoring on water quality in a subtropical reservoir by incorporating grammatical evolution with parallel genetic algorithms into satellite imagery, Water Research **42**, pp. 296-306, 2008.
21. J. Tavares, F.B. Pereira, Automatic Design of Ant Algorithms with Grammatical Evolution. In: Moraglio, A., Silva, S., Krawiec, K., Machado, P., Cotta, C. (eds) Genetic Programming. EuroGP 2012. Lecture Notes in Computer Science, vol 7244. Springer, Berlin, Heidelberg, 2012.
22. C. Ryan, M. O'Neill, J.J. Collins, Grammatical evolution: Solving trigonometric identities, proceedings of Mendel. Vol. 98. 1998.
23. A.O. Puente, R. S. Alfonso, M. A. Moreno, Automatic composition of music by means of grammatical evolution, In: APL '02: Proceedings of the 2002 conference on APL: array processing languages: lore, problems, and applications July 2002 Pages 148–155.
24. Lídio Mauro Limade Campo, R. Célio Limã Oliveira,Mauro Roisenberg, Optimization of neural networks through grammatical evolution and a genetic algorithm, Expert Systems with Applications **56**, pp. 368-384, 2016.
25. K. Soltanian, A. Ebnenasir, M. Afsharchi, Modular Grammatical Evolution for the Generation of Artificial Neural Networks, Evolutionary Computation **30**, pp 291–327, 2022.
26. E. Galván-López, J.M. Swafford, M. O'Neill, A. Brabazon, Evolving a Ms. PacMan Controller Using Grammatical Evolution. In: , et al. Applications of Evolutionary Computation. EvoApplications 2010. Lecture Notes in Computer Science, vol 6024. Springer, Berlin, Heidelberg, 2010.
27. N. Shaker, M. Nicolau, G. N. Yannakakis, J. Togelius, M. O'Neill, Evolving levels for Super Mario Bros using grammatical evolution, 2012 IEEE Conference on Computational Intelligence and Games (CIG), 2012, pp. 304-31.
28. D. Martínez-Rodríguez, J. M. Colmenar, J. I. Hidalgo, R.J. Villanueva Micó, S. Salcedo-Sanz, Particle swarm grammatical evolution for energy demand estimation, Energy Science and Engineering **8**, pp. 1068-1079, 2020.
29. N. R. Sabar, M. Ayob, G. Kendall, R. Qu, Grammatical Evolution Hyper-Heuristic for Combinatorial Optimization Problems, IEEE Transactions on Evolutionary Computation **17**, pp. 840-861, 2013.
30. C. Ryan, M. Kshirsagar, G. Vaidya, G. et al. Design of a cryptographically secure pseudo random number generator with grammatical evolution. Sci Rep **12**, 8602, 2022.
31. P.J. Pereira, P. Cortez, R. Mendes, Multi-objective Grammatical Evolution of Decision Trees for Mobile Marketing user conversion prediction, Expert Systems with Applications **168**, 114287, 2021.
32. F. Castejón, E.J. Carmona, Automatic design of analog electronic circuits using grammatical evolution, Applied Soft Computing **62**, pp. 1003-1018, 2018.
33. L. Araujo, J. Martinez-Romo, A. Duque, Discovering taxonomies in Wikipedia by means of grammatical evolution. Soft Comput **22**, pp. 2907–2919, 2018.
34. C. Martín, D. Quintana, P. Isasi, Grammatical Evolution-based ensembles for algorithmic trading, Applied Soft Computing **84**, 105713, 2019.
35. Moore, J.H., Sipper, M. (2018). Grammatical Evolution Strategies for Bioinformatics and Systems Genomics. In: Ryan, C., O'Neill, M., Collins, J. (eds) Handbook of Grammatical Evolution. Springer, Cham. https://doi.org/10.1007/978-3-319-78717-6_16
36. Hidalgo, J. I., Colmenar, J. M., Risco-Martin, J. L., Cuesta-Infante, A., Maqueda, E., Botella, M., & Rubio, J. A. (2014). Modeling glycemia in humans by means of grammatical evolution. Applied Soft Computing, 20, 40-53.
37. Tsoulos, I. G. (2020). Creating classification rules using grammatical evolution. International Journal of Computational Intelligence Studies, 9(1-2), 161-171.
38. Anastasopoulos, N.; Tsoulos, I.G.; Tzallas, A. GenClass: A parallel tool for data classification based on Grammatical Evolution. SoftwareX 2021, 16, 100830.
39. Spyrou, E. D., Stylios, C., & Tsoulos, I. (2023). Classification of CO Environmental Parameter for Air Pollution Monitoring with Grammatical Evolution. Algorithms, 16(6), 300.
40. Margariti, S. V., Tsoulos, I. G., Kiousi, E., & Stergiou, E. (2024). Traffic Classification in Software-Defined Networking Using Genetic Programming Tools. Future Internet, 16(9), 338.
41. M. Kelly, R. Longjohn, K. Nottingham, The UCI Machine Learning Repository, https://archive.ics.uci.edu.
42. J. Alcalá-Fdez, A. Fernandez, J. Luengo, J. Derrac, S. García, L. Sánchez, F. Herrera. KEEL Data-Mining Software Tool: Data Set Repository, Integration of Algorithms and Experimental Analysis Framework. Journal of Multiple-Valued Logic and Soft Computing 17, pp. 255-287, 2011.
43. Tzimourta, K.D.; Tsoulos, I.; Bilero, I.T.; Tzallas, A.T.; Tsipouras, M.G.; Giannakeas, N. Direct Assessment of Alcohol Consumption in Mental State Using Brain Computer Interfaces and Grammatical Evolution. Inventions 2018, 3, 51.
44. Weiss, Sholom M. and Kulikowski, Casimir A., Computer Systems That Learn: Classification and Prediction Methods from Statistics, Neural Nets, Machine Learning, and Expert Systems, Morgan Kaufmann Publishers Inc, 1991.
45. Banu PK, N. (2024). Feature Engineering for Epileptic Seizure Classification Using SeqBoostNet. International Journal of Computing and Digital Systems, 16(1), 1-14.
46. G. Demiroz, H.A. Govenir, N. Ilter, Learning Differential Diagnosis of Eryhemato-Squamous Diseases using Voting Feature Intervals, Artificial Intelligence in Medicine. **13**, pp. 147–165, 1998.

47. L. Prechelt, "PROBEN1—A Set of Neural Network Benchmark Problems and Benchmarking Rules," Technical Report 21/94, Fakultät für Informatik, Universität Karlsruhe, Germany, 1994.
48. B. Hayes-Roth, B., F. Hayes-Roth. Concept learning and the recognition and classification of exemplars. Journal of Verbal Learning and Verbal Behavior **16**, pp. 321-338, 1977.
49. Hepatitis [Data set]. (1983). UCI Machine Learning Repository. https://doi.org/10.24432/C5Q59J
50. Congressional Voting Records [Data set]. (1987). UCI Machine Learning Repository. https://doi.org/10.24432/C5C01P
51. J.G. Dy , C.E. Brodley, Feature Selection for Unsupervised Learning, The Journal of Machine Learning Research **5**, pp 845–889, 2004.
52. S. J. Perantonis, V. Virvilis, Input Feature Extraction for Multilayered Perceptrons Using Supervised Principal Component Analysis, Neural Processing Letters **10**, pp 243–252, 1999.
53. G. Cestnik, I. Konenenko, I. Bratko, Assistant-86: A Knowledge-Elicitation Tool for Sophisticated Users. In: Bratko, I. and Lavrac, N., Eds., Progress in Machine Learning, Sigma Press, Wilmslow, pp. 31-45, 1987.
54. Olave, M., Rajkovic, V., & Bohanec, M. (1989). An application for admission in public school systems. Expert Systems in Public Administration, 1, 145-160.
55. Kaynak, C. (1995). Methods of combining multiple classifiers and their applications to handwritten digit recognition. Unpublished master's thesis, Bogazici University.
56. E. Alpaydin and F. Alimoglu, "Pen-Based Recognition of Handwritten Digits," UCI Machine Learning Repository, 1998, doi: 10.24432/C5MG6K.
57. A. Guerin-Dugué et al., "Deliverable R3-B4-P — Task B4: Benchmarks," Technical Report, ELENA-NERVES II, ESPRIT Basic Research Project 6891, 1995.
58. Emad-Ud-Din, M., & Wang, Y. (2023). Promoting occupancy detection accuracy using on-device lifelong learning. IEEE Sensors Journal, 23(9), 9595-9606.
59. A. Srinivasan, "Statlog (Landsat Satellite)," UCI Machine Learning Repository, 1993, doi: 10.24432/C55887.
60. M. Dash, H. Liu, P. Scheuermann, K. L. Tan, Fast hierarchical clustering and its validation, Data & Knowledge Engineering **44**, pp 109–138, 2003.
61. Statlog (Shuttle) [Data set]. (n.d.). UCI Machine Learning Repository. https://doi.org/10.24432/C5WS31
62. M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, "Spambase," UCI Machine Learning Repository, 1999, doi: 10.24432/C53G6X.
63. Gavrilis, D., Tsoulos, I. G., & Dermatas, E. (2008). Selecting and constructing features using grammatical evolution. Pattern Recognition Letters, 29(9), 1358–1365. https://doi.org/10.1016/j.patrec.2008.02.007
64. P. Cortez, A. M. Gonçalves Silva, Using data mining to predict secondary school student performance, In Proceedings of 5th FUture BUsiness TEChnology Conference (FUBUTEC 2008) (pp. 5–12). EUROSIS-ETI, 2008.
65. Molecular Biology (Splice-junction Gene Sequences) [Data set]. (1991). UCI Machine Learning Repository. https://doi.org/10.24432/C5M888
66. KEEL Project. (n.d.). Texture [Data set]. University of Granada. https://sci2s.ugr.es/keel/dataset.php?cod=72
67. R. Quinlan, "Thyroid Disease," UCI Machine Learning Repository, 1986, doi: 10.24432/C5D010.
68. L. Breiman, "Bias, Variance, and Arcing Classifiers," Technical Report 460, Statistics Department, University of California, Berkeley, 1996.
69. J. P. Siebert, "Vehicle Recognition Using Rule Based Methods," Turing Institute Research Memorandum TIRM-87-018, Turing Institute, Glasgow, UK, March 1987.
70. P. Horton and K. Nakai, "A Probabilistic Classification System for Predicting the Cellular Localization Sites of Proteins," Proceedings of the Fourth International Conference on Intelligent Systems for Molecular Biology (ISMB), pp. 109–115, 1996.
71. R. G. Andrzejak, K. Lehnertz, F.Mormann, C. Rieke, P. David, and C. E. Elger, "Indications of nonlinear deterministic and finite-dimensional structures in time series of brain electrical activity: dependence on recording region and brain state," Physical Review E, vol. 64, no. 6, Article ID 061907, 8 pages, 2001.
72. A. T. Tzallas, M. G. Tsipouras, and D. I. Fotiadis, "Automatic Seizure Detection Based on Time-Frequency Analysis and Artificial Neural Networks," Computational Intelligence and Neuroscience, vol. 2007, Article ID 80510, 13 pages, 2007. doi:10.1155/2007/80510
73. J. Park and I. W. Sandberg, Universal Approximation Using Radial-Basis-Function Networks, Neural Computation **3**, pp. 246-257, 1991.
74. G.A. Montazer, D. Giveki, M. Karami, H. Rastegar, Radial basis function neural networks: A review. Comput. Rev. J **1**, pp. 52-74, 2018.
75. K. O. Stanley, R. Miikkulainen, Evolving Neural Networks through Augmenting Topologies, Evolutionary Computation **10**, pp. 99-127, 2002.

76. Ward, R., Wu, X., & Bottou, L. (2020). Adagrad stepsizes: Sharp convergence over nonconvex landscapes. Journal of Machine Learning Research, 21(219), 1-30.
77. D. P. Kingma, J. L. Ba, ADAM: a method for stochastic optimization, in: Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015), pp. 1–15, 2015.
78. M.J.D Powell, A Tolerant Algorithm for Linearly Constrained Optimization Calculations, Mathematical Programming **45**, pp. 547-566, 1989.